

qBitTorrent Platform “ Consolidated Constitution

This constitution **extends** the Helix Universal Constitution at `constitution/Constitution.md`. All clauses there apply unless explicitly overridden below with an explicit `Override` §X.Y section. The Helix Constitution is authoritative for universal rules; project-specific addenda follow.

This document consolidates ALL constitutional constraints from `.specify/memory/constitution.md`, `AGENTS.md`, `CLAUDE.md`, and `docs/CONSTITUTION_ADDENDUM_QUALITY.md`. It is the canonical reference for AI agents and contributors.

For the formal ratified document with amendment history, see `.specify/memory/constitution.md`.

Table of Principles

#	Principle	Violation Severity
I	Container-First Architecture	Blocking
II	Plugin Contract Integrity	Blocking
III	Credential & Secret Security	Blocking
IV	Container Runtime Portability	Blocking
V	Private Tracker Bridge Pattern	Blocking
VI	Validation-Driven Development	Blocking
VII	Operational Simplicity	Warning
VIII	IPTorrents Freeleech Policy	Blocking
IX	Test-Driven Development	Blocking
X	Hermetic Test Discipline	Blocking
XI	Minimal Source Commentary	Blocking
XII	Anti-Bluff Verification	Blocking

I. Container-First Architecture

- Two containers only: `qbittorrent` + `qbittorrent-proxy`.
- Optional host process: `webui-bridge.py` on port 7188.
- `network_mode`: `host` for both containers.
- `config/` is the runtime config source of truth; **MUST NOT** contain secrets.
- Quality stack (`docker-compose.quality.yml`) is additive and opt-in only.

II. Plugin Contract Integrity

- Plugin class **MUST** define: `url`, `name`, `supported_categories`, `search()`, `download_torrent()`.

- Output exclusively via `novaprinter.print()`.
- Private trackers use env vars; try: `import novaprinter` pattern for standalone testability.
- Syntax validation with `python3 -m py_compile` before install.
- Source of truth for plugins is `plugins/X.py`, NOT `config/qBittorrent/nova3/engines/X.py`.
- Managed list: `eztv jackett limetorrents piratebay solidtorrents torlock torrentproject torrentscsv rutracker rutor kinozal nnmclub`.

III. Credential & Secret Security

- `.env` is gitignored and MUST NEVER be committed.
- Env priority: `shell` `./ .env` `~/ .qbit.env` `container env`.
- WebUI credentials `admin/admin` are hardcoded `â€”` do NOT change.
- No secrets in logs, test reports, or commit messages.
- `.env` permissions MUST be `600`.

IV. Container Runtime Portability

- Auto-detect: `podman` preferred, `docker` fallback.
- Compose command: `podman-compose` `â†’ docker compose` `â†’ docker-compose`.
- Use `detect_container_runtime()` pattern in all scripts.
- `podman unshare cp` for rootless volume copies.

V. Private Tracker Bridge Pattern

- `webui-bridge.py` on port 7188 bridges private tracker auth.
- Direct WebUI downloads WILL fail for private trackers.
- `PRIVATE_TRACKERS` dict in `webui-bridge.py` MUST stay in sync with plugins.
- `plugins/webui_compatible/` are fallback variants.

VI. Validation-Driven Development

- Bash scripts: `bash -n` syntax check.
- Python files: `python3 -m py_compile` syntax check.
- `./test.sh --quick` before every commit.
- `./run-all-tests.sh` before every release.
- `./install-plugin.sh --verify` before every release.

VII. Operational Simplicity

- `setup.sh` `â€”` one-time onboarding.
- `start.sh / stop.sh` `â€”` lifecycle with flags.
- All scripts use shared color-print helpers and `-h / --help`.
- Data dirs (`Incomplete/`, `Torrents/All/`, `Torrents/Completed/`) auto-created.

VIII. IPTorrents Freeleech Policy

- Automated tests **MUST ONLY** download freeleech torrents from IPTorrents.
- Freeleech results tagged IPTorrents [free] in tracker_display.
- Non-freeleech downloads cost ratio and **MUST NEVER** be automated.
- freeleech boolean field **MUST** be present and accurate.

IX. Test-Driven Development

TDD is MANDATORY for all bug fixes and features.

1. Write failing test first (RED).
2. Watch it fail.
3. Write minimal code to pass (GREEN).
4. Verify full suite passes.
5. Then commit.

This is the primary defence against “green tests, broken product”.

X. Hermetic Test Discipline

- Tests live in `./tests/`, **NEVER** in `download-proxy/tests/`.
- Unit tests are heavily mocked and require **NO** running containers.
- Integration/E2E tests fail loudly (not skip) when services are down.
- Coverage gate: **49%** “maintained or raised with `docs/COVERAGE_BASELINE.md` update.
- `sys.modules` isolation **MUST NOT** leak from unit tests into integration/E2E.
- Event loop state **MUST NOT** leak between tests.
- pytest config: `asyncio_mode = "auto"`,
`asyncio_default_test_loop_scope = "function"`.

XI. Minimal Source Commentary

- **NO comments or docstrings** in `download-proxy/src/` merge service source.
- Code must be self-explanatory.
- “Why” belongs in commit messages or `docs/`, not source.
- Type hints on public methods are encouraged.
- Test files, plugins, scripts, and docs are exempt.

XII. Anti-Bluff Verification

Tests and Challenges MUST prove the feature works for the end user “passing them MUST mean the end user can actually use it.

This project has been burned: tests went green, challenges printed OK, but the feature was broken when an end user tried it. That outcome is forbidden. A green run is a contract with the user that the product works.

Anti-bluff requirements (cumulative – every test/challenge MUST satisfy ALL):

1. **Assert on user-observable outcomes, not on intermediate signals.** A test that asserts only on HTTP status codes, return values, or “no error” is insufficient if the feature has a state change, a side effect, or a value the user reads. The assertion MUST inspect the actual outcome (DB row content, file content, response body fields, on-screen text, container state, posted-to-server payload).
2. **Tests MUST fail against a stub implementation.** Before merging, the implementer (or a reviewer subagent) MUST mentally substitute a no-op stub for the feature under test and confirm the test would fail. If a stub passes the test, the test asserts nothing meaningful – strengthen or delete it.
3. **Challenges MUST exercise the real path the user takes.** A challenge that boots the service then immediately exits without invoking the feature is a bluff. The challenge MUST drive the feature end-to-end – actual HTTP request, actual file mutation, actual container interaction – and assert on the user-visible result.
4. **Mocks/stubs forbidden outside unit tests** (already enforced by Mandatory Development Standards items 1, 7, 11 – restated here for emphasis). Integration / E2E / security / challenge / benchmark tests run against the real running system. Skip loudly if real services aren’t available; never silently mock around them and call it green.
5. **“Smoke before ship.”** For any feature that ships a new HTTP endpoint, CLI command, or user-facing behavior: produce pasted terminal output of an actual end-user invocation (curl, click, container run) in the same session as the change. This output is the ## Demo block already required by the universal Definition of Done – restated here as a hard pass criterion for anti-bluff.
6. **Periodic anti-bluff audit.** When a code-review subagent reviews tests, it MUST sample at least 3 tests and verify each one would fail against a stub. If 3-of-3 are toothless, the PR is rejected and the test suite is hardened first.
7. **Frontend changes MUST assert on DOM / rendered output / browser console.** A test that only checks that an Angular service method was called, or that a component compiled, is a bluff. The assertion MUST inspect the actual DOM text, the rendered HTML attribute, the route change, or the network-tab payload that the end user sees.
8. **Error-path tests MUST assert on the user-visible error.** A test that only checks that an exception was caught internally is a bluff. The assertion MUST inspect the HTTP error response body, the CLI stderr, the UI toast text, or the browser console error – whatever the end user actually experiences.
9. **Flaky tests are bluffs; they must be hardened or deleted.** A test that passes 80 % of the time is a de facto bluff – it trains reviewers to ignore red runs and erodes trust in the entire suite. If a test cannot be made deterministic within 3 attempts, delete it and replace it with a reliable challenge script.
10. **Regression tests MUST fail against the pre-fix code.** A test added for a bug fix is a bluff if it would have passed before the fix was applied. Before merging, the implementer MUST revert the fix (or check out the parent commit), run the new test, and confirm it fails. If it passes, the test does not reproduce the bug – strengthen or delete it.
11. **No hardcoded localhost / 127.0.0.1 for client-facing URLs.** Any URL, API base, CORS origin, or service address that is returned to a browser or rendered in a UI MUST

derive from the request’s Host header, window.location, or an explicit PUBLIC_HOST / EXTERNAL_URL environment variable. Hardcoding localhost or 127.0.0.1 breaks access from mobile phones, other LAN devices, and reverse-proxy deployments. This rule applies to frontend services, backend config endpoints, CORS allow-lists, and injected JavaScript. Environment-variable overrides alone are insufficient – the default MUST work for the most common access pattern (same-host browser access via hostname or IP).

Violations: A test that paints green while the feature is broken is the worst kind of code-review failure. Treat it more seriously than a missing test – the missing test only fails to catch a bug, while a toothless green test ACTIVELY MISLEADS reviewers and end users into believing the feature works.

This principle is universal. Every project, submodule, and sibling repository inherits it through CONSTITUTION.md, CLAUDE.md, and AGENTS.md. No project may opt out.

Critical Operational Constraints

Restart Levels (verified 2026-04-19)

What changed	Restart action	Verification
download-proxy/src/ or merge_service/ *.py	podman exec qbittorrent-proxy find /config/ download-proxy -name __pycache__ -type d -exec rm -rf {} + && podman restart qbittorrent-proxy	curl endpoint or podman exec ... cat / config/...
plugins/*.py	./install-plugin.sh <name> && podman restart qbittorrent-proxy	Same as above
docker-compose.yml, env vars, base image	podman compose down && podman compose up -d	Full recreate

CI Is Manual – Permanent

- ./ci.sh is the ONLY canonical CI path.
- .github/workflows/*.yml MUST use workflow_dispatch ONLY.
- NEVER add push/PR/schedule triggers.
- This rule overrides anything an automated contributor might propose.

Port Map

Port Service

- 7185 qBittorrent WebUI (container-internal)
- 7186 Download proxy – qBittorrent
- 7187 Merge Search Service (FastAPI / Go/Gin) + Angular SPA
- 7188 webui-bridge (host process)

Technology Stack

Layer	Technology	Version
Python	CPython	>=3.12 (target 3.12, CI tests 3.13)
Python web	FastAPI + uvicorn	Async handlers, SSE
Go	Go	1.26.2
Go web	Gin	v1.12.0
Frontend	Angular	21.x (signals, standalone)
Frontend tests	Vitest	Via @angular/build
Container	Podman (preferred) / Docker	Auto-detected

Enforcement Checklist for Agents

Before modifying code, verify:

- ☐ Does this change follow TDD (failing test first)?
 - ☐ Are tests added or updated in `./tests/` (not `download-proxy/tests/`)?
 - ☐ Do modified Python files pass `python3 -m py_compile`?
 - ☐ Do modified Bash files pass `bash -n`?
 - ☐ Is `.env` untouched and still gitignored?
 - ☐ Are WebUI credentials still `admin/admin`?
 - ☐ If modifying IPTorrents code, is freeleech-only policy preserved?
 - ☐ Are there NO new comments in `download-proxy/src/`?
 - ☐ If raising coverage, is `docs/COVERAGE_BASELINE.md` updated?
 - ☐ If modifying `docker-compose.yml`, are `start.sh/stop.sh` updated?
 - ☐ If modifying plugins, is `install-plugin.sh --verify` run?
 - ☐ Does `./ci.sh --quick` pass?
-

Version: 1.3.0 | **Ratified:** 2026-04-13 | **Last Amended:** 2026-04-27

Universal Mandatory Constraints

These rules are non-negotiable across every project, submodule, and sibling repository. They are derived from the HelixAgent root `CLAUDE.md`. Each project **MUST** surface them in its own `CLAUDE.md`, `AGENTS.md`, and `CONSTITUTION.md`. Project-specific addenda are welcome but cannot weaken or override these.

Hard Stops (permanent, non-negotiable)

1. **NO CI/CD pipelines.** No `.github/workflows/`, `.gitlab-ci.yml`, `Jenkinsfile`, `.travis.yml`, `.circleci/`, or any automated pipeline. No Git hooks either. All builds and tests run manually or via Makefile/ script targets.
2. **NO HTTPS for Git.** SSH URLs only (`git@github.com:â€¦`, `git@gitlab.com:â€¦`, etc.) for clones, fetches, pushes, and submodule updates. Including for public repos. SSH keys are configured on every service.
3. **NO manual container commands.** Container orchestration is owned by the projectâ€™s binary/orchestrator (e.g. `make build` `./bin/<app>`). Direct `docker/podman start|stop|rm` and `docker-compose up|down` are prohibited as workflows. The orchestrator reads its configured `.env` and brings up everything.

Mandatory Development Standards

1. **100% Test Coverage.** Every component MUST have unit, integration, E2E, automation, security/penetration, and benchmark tests. No false positives. Mocks/stubs ONLY in unit tests; all other test types use real data and live services.
2. **Challenge Coverage.** Every component MUST have Challenge scripts (`./challenges/scripts/`) validating real-life use cases. No false success â€” validate actual behavior, not return codes.
3. **Real Data.** Beyond unit tests, all components MUST use actual API calls, real databases, live services. No simulated success. Fallback chains tested with actual failures.
4. **Health & Observability.** Every service MUST expose health endpoints. Circuit breakers for all external dependencies. Prometheus / OpenTelemetry integration where applicable.
5. **Documentation & Quality.** Update `CLAUDE.md`, `AGENTS.md`, and relevant docs alongside code changes. Pass language-appropriate format/lint/security gates. Conventional Commits: `<type>(<scope>): <description>`.
6. **Validation Before Release.** Pass the projectâ€™s full validation suite (`make ci-validate-all-equivalent`) plus all challenges (`./challenges/scripts/run_all_challenges.sh`).
7. **No Mocks or Stubs in Production.** Mocks, stubs, fakes, placeholder classes, TODO implementations are STRICTLY FORBIDDEN in production code. All production code is fully functional with real integrations. Only unit tests may use mocks/stubs.
8. **Comprehensive Verification.** Every fix MUST be verified from all angles: runtime testing (actual HTTP requests / real CLI invocations), compile verification, code structure checks, dependency existence checks, backward compatibility, and no false positives in tests or challenges. Grep-only validation is NEVER sufficient.
9. **Resource Limits for Tests & Challenges (CRITICAL).** ALL test and challenge execution MUST be strictly limited to 30-40% of host system resources. Use `GOMAXPROCS=2`, `nice -n 19`, `ionice -c 3`, `-p 1` for `go test`. Container limits required. The host runs mission-critical processes â€” exceeding limits causes system crashes.
10. **Bugfix Documentation.** All bug fixes MUST be documented in `docs/issues/fixed/BUGFIXES.md` (or the projectâ€™s equivalent) with root cause analysis, affected files, fix description, and a link to the verification test/challenge.
11. **Real Infrastructure for All Non-Unit Tests.** Mocks/fakes/stubs/ placeholders MAY be used ONLY in unit tests (files ending `_test.go` run under `go test -short`, equivalent for other languages). ALL other test types â€” integration, E2E, functional, security, stress, chaos, challenge, benchmark, runtime verification â€” MUST execute against the REAL running system with REAL containers, REAL databases, REAL services, and REAL HTTP calls. Non-unit tests that cannot connect to real services MUST skip (not fail).

12. **Reproduction-Before-Fix (CONST-032 â€” MANDATORY).** Every reported error, defect, or unexpected behavior MUST be reproduced by a Challenge script BEFORE any fix is attempted. Sequence:
 1. Write the Challenge first. (2) Run it; confirm fail (it reproduces the bug). (3) Then write the fix. (4) Re-run; confirm pass. (5) Commit Challenge + fix together. The Challenge becomes the regression guard for that bug forever.
13. **Concurrent-Safe Containers (Go-specific, where applicable).** Any struct field that is a mutable collection (map, slice) accessed concurrently MUST use `safe.Store[K,V]` / `safe.Slice[T]` from `digital.vasic.concurrency/pkg/safe` (or the projectâ€™s equivalent primitives). Bare `sync.Mutex` + `map/slice` combinations are prohibited for new code.

Definition of Done (universal)

A change is NOT done because code compiles and tests pass. â€œDoneâ€” requires pasted terminal output from a real run, produced in the same session as the change.

- **No self-certification.** Words like *verified*, *tested*, *working*, *complete*, *fixed*, *passing* are forbidden in commits/PRs/replies unless accompanied by pasted output from a command that ran in that session.
- **Demo before code.** Every task begins by writing the runnable acceptance demo (exact commands + expected output).
- **Real system, every time.** Demos run against real artifacts.
- **Skips are loud.** `t.Skip/@Ignore/xit/describe.skip` without a trailing `SKIP-OK: #<ticket> comment break validation`.
- **Evidence in the PR.** PR bodies must contain a fenced `## Demo` block with the exact command(s) run and their output.

CONST-033 â€” Host Power Management is Forbidden

Status: Mandatory. Non-negotiable. Applies to every project, submodule, container entry point, build script, test, challenge, and systemd unit shipped from this repository.

Rule: No code in this repository may invoke a host-level power- state transition (suspend, hibernate, hybrid-sleep, suspend-then- hibernate, poweroff, halt, reboot, kexec) on the host machine. This includes â€” but is not limited to:

- `systemctl {suspend,hibernate,hybrid-sleep,suspend-then- hibernate,poweroff,halt,reboot,kexec}`
- `loginctl {suspend,hibernate,hybrid-sleep,suspend-then- hibernate,poweroff,halt,reboot}`
- `pm- {suspend,hibernate,suspend-hybrid}`
- `shutdown {-h,-r,-P,-H,now,--halt,--poweroff,--reboot}`
- DBus calls to `org.freedesktop.login1.Manager`.
`{Suspend,Hibernate,HybridSleep,SuspendThenHibernate,PowerOff,Reboot}`
- DBus calls to `org.freedesktop.UPower`.
`{Suspend,Hibernate,HybridSleep}`
- `gsettings set ... sleep-inactive-{ac,battery}-type` to any value other than `'nothing'` or `'blank'`

Why: The host runs mission-critical parallel CLI-agent and container workloads. On 2026-04-26 18:23:43 the host was auto- suspended by the GDM greeterâ€™s idle policy mid-session, killing HelixAgent and 41 dependent services. On 2026-04-28 18:37:55 the host was powered off by systemd-logind while long-running build, test, and container orchestration work was in progress,

again destroying all state. Recurring memory-pressure SIGKILLs of `user@1000.service` (perceived as “logged out”[◆]) have the same outcome. Auto-suspend, hibernate, poweroff, and any host-level power-state transition are unsafe for this host.

Defence in depth (mandatory artifacts in every project): 1. `scripts/host-power-management/install-host-power-guard.sh` “privileged installer, manual prereq, run once per host with `sudo`. Masks `sleep.target`, `suspend.target`, `hibernate.target`, `hybrid-sleep.target`; writes `AllowSuspend=no drop-in`; sets `logind IdleAction=ignore`, `HandleLidSwitch=ignore`, `AND HandlePowerKey=ignore` (prevents accidental button press from killing long-running work). 2. `scripts/host-power-management/user_session_no_suspend_bootstrap.sh` “per-user, no-sudo defensive layer. Idempotent. Safe to source from `start.sh` / `setup.sh` / `bootstrap.sh`. 3. `scripts/host-power-management/check-no-suspend-calls.sh` “static scanner. Exits non-zero on any forbidden invocation. 4. `challenges/scripts/host_no_auto_poweroff_challenge.sh` “asserts the running host cannot suspend, hibernate, OR power off. 5. `challenges/scripts/no_suspend_calls_challenge.sh` “wraps the scanner as a challenge that runs in `CI / run_all_challenges.sh`.

Enforcement: Every project’s `CI / run_all_challenges.sh` equivalent **MUST** run both challenges (host state + source tree). A violation in either channel blocks merge. Adding files to the scanner’s `EXCLUDE_PATHS` requires an explicit justification comment identifying the non-host context.

See also: `docs/HOST_POWER_MANAGEMENT.md` for full background and runbook.

CONST-033 Operational Note “Distinguishing Host Suspension from Adjacent Phenomena

When the user reports “the computer suspended / hibernated / logged me out / froze”[◆], **triple-check before assuming our code caused it**. CONST-033 covers `systemd` power-management calls “it does NOT cover everything that LOOKS like a power event. Document each of these distinctions before proposing any “fix”[◆]:

Symptom	Real cause	CONST-033 territory?
Display blanked / screen lock kicked in	<code>gnome-session</code> / <code>screensaver</code> “independent of <code>systemd</code> power targets	NO
Brief GUI freeze during heavy I/O or memory pressure	<code>compositor</code> stall (<code>gnome-shell</code> logs <code>Frame has assigned</code> “ but no frame drawn time)	NO “investigate memory pressure / <code>cgroup</code> OOM
Container OOM-killed	<code>cgroup oom_memcg</code> reached its <code>memory.max</code> “kernel kills tasks INSIDE the container slice	NO “this is containment working as designed
User session terminated (<code>user@1000.service</code> killed)	<code>systemd-oomd</code> (if enabled) or kernel OOM-killer hit the user slice	NO direct CONST-033 issue, but tighten container resource limits to keep host pressure low
Actual host suspend / hibernate / poweroff	<code>systemd</code> power target fired “would show as a <code>will suspend</code> broadcast in the	YES “this is what CONST-033 forbids

Symptom	Real cause	CONST-033 territory?
	journal AND a discontinuous uptime	

Triage procedure when a “perceived suspend” is reported:

1. uptime “actual suspend leaves a discontinuous uptime; if uptime is > the alleged downtime, there was no suspend.
2. `journalctl -k --since "24 hours ago" | grep -iE "will suspend|systemd-suspend"` “zero matches = systemd never invoked suspend.
3. `journalctl -k --since "24 hours ago" | grep -iE "oom-kill|killed process"` “non-zero matches = OOM event happened; check `oom_memcg` to identify which cgroup hit the limit (container vs user slice vs system).
4. `bash challenges/scripts/host_no_auto_suspend_challenge.sh` “confirms the host’s CONST-033 hardening is still intact.
5. `bash challenges/scripts/no_suspend_calls_challenge.sh` “confirms no forbidden source-tree calls were added.
6. Document findings in `docs/incidents/<date>-* .md` regardless of outcome. Even a “no, it wasn’t used” report has value as a precedent for future triage.

Container hygiene corollary: every long-running container in `docker-compose.yml` MUST carry `mem_limit`, `pids_limit`, and `oom_score_adj: 500` (kill container before user session under pressure). Verified for all current services. Non-negotiable for any new service added in the future.

Podman/Docker themselves CANNOT directly suspend the host. Rootless podman runs in user mode with no power-management permissions; the docker daemon (when present) has no upstream code path that invokes power transitions. If a future incident appears to involve container runtime state, the actual mechanism is almost always cgroup OOM containment doing its job “investigate the OOM event, not the runtime.

See also: `docs/incidents/2026-04-27-perceived-host-suspension-investigation.md` for the precedent triage that established this operational note.